

Binárne rozhodovacie diagramy

Dátové štruktúry a algoritmy.

Dokumentácia

1 Opis riešenia

Implementácia je tvorená jedným hlavičkovým súborom `bdd.h` a jedným zdrojovým súborom `bdd.c`. Celé riešenie možno rozdeliť do troch logických vrstiev, ktoré na seba nadväzujú: parser a evaluátor booleovských výrazov, unique tabuľka slúžiaca na zdieľanie uzlov, a samotné rekurzívne budovanie BDD pomocou Shannonovej expanzie.

Vstupom funkcie `BDD_create` je reťazec reprezentujúci booleovský výraz v disjunktívnej normálnej forme (DNF) a reťazec reprezentujúci požadované poradie premenných. Výraz je spracovaný priebežne počas rekurzívneho konštruovania grafu. Na každej úrovni rekurzcie sa výraz zjednoduší dosadením aktuálnej premennej a odovzdá sa nižšej úrovni. Vďaka tomu zostáva kód kompaktný a priamo prepojený so štruktúrou BDD.

Kľúčovým prvkom správnosti a efektivity riešenia je **unique tabuľka** (hash mapa), ktorá zabezpečuje, že každý uzol s rovnakou trojicou (premenná, high-potomok, low-potomok) existuje v grafe práve raz. Spolu s pravidlom eliminácie uzlov (ak sú obaja potomkovia zhodní, uzol sa vynechá) tieto dve redukčné pravidlá zaručujú, že výsledkom je redukovaný BDD.

2 Vlastné dátové štruktúry

2.1 BDDNode

Štruktúra `BDDNode` reprezentuje jeden uzol v grafe BDD. Každý uzol môže byť buď **vnútorný** (rozhodovací) alebo **terminálny** (listový). Vnútorný uzol nesie premennú, podľa ktorej sa rozvetvuje, a dva ukazovatele na potomkov: `high` pre prípad, že dosadzovaná premenná má hodnotu 1, a `low` pre hodnotu 0. Terminálny uzol má pole `variable` nastavené na `'\0'` a nesie len celočíselnú hodnotu 0 alebo 1.

```
typedef struct BDDNode {
    char        variable; /* premenná uzla, '\0' = terminál */
    int         value;    /* 0 alebo 1, platné len pre terminál */
    struct BDDNode *high; /* hrana pre variable = 1 */
    struct BDDNode *low;  /* hrana pre variable = 0 */
    int         id;       /* unikátne ID (pre ladenie) */
} BDDNode;
```

Pole `id` je čisto pomocné a pri ladení umožňuje rozlíšiť uzly, ktoré by inak bolo ťažké identifikovať len podľa obsahu. Terminálne uzly dostávajú pevné ID `-1` a `-2`, čím sa ľahko odlišia od vnútorných uzlov počítaných od 0.

2.2 Utnode a UniqueTable

Unique tabuľka je implementovaná ako hash mapa s reťazením (chaining). Kľúčom každého záznamu je trojica (`variable`, `high`, `low`). Štruktúra `Utnode` tvorí jeden záznam v reťazci (bucket-e) a okrem kľúčových polí uchováva ukazovateľ na príslušný `BDDNode` a ukazovateľ na ďalší záznam v reťazci.

```

typedef struct UTNode {
    char        variable;
    BDDNode     *high;
    BDDNode     *low;
    BDDNode     *node;    /* uložený uzol */
    struct UTNode *next;
} UTNode;

typedef struct UniqueTable {
    UTNode      **buckets;
    long long    size;     /* aktuálny počet bucketov */
    long long    count;    /* počet vložených záznamov */
} UniqueTable;

```

Tabuľka sa inicializuje s `UT_INIT_SIZE = 256` bucketmi. Keď obsadenosť (load factor) presiahne hodnotu `UT_ALPHA_HIGH = 0.75`, spustí sa rehashing, kapacita sa zdvojnásobí a všetky záznamy sa pre rozmiestnia. Vďaka tomu zostáva priemerný počet prvkov v jednom buckete blízky 1, čo zaručuje amortizovane konštantný čas vyhľadávania a vkladania..

2.3 BDD

Štruktúra `BDD` je obalová štruktúra celého grafu. Uchováva ukazateľ na koreňový uzol, oba zdieľané terminálne uzly, počet premenných a uzlov, referenciu na unique tabuľku a poradie premenných ako reťazec znakov.

```

typedef struct BDD {
    BDDNode     *root;
    BDDNode     *terminal0; /* zdieľaný terminál 0 */
    BDDNode     *terminal1; /* zdieľaný terminál 1 */
    int         numVariables;
    int         numNodes;   /* počet ne-terminálnych uzlov */
    UniqueTable *ut;
    char        *varOrder;  /* napr. "BAC" */
} BDD;

```

Terminálne uzly sú alokované priamo v `BDD_create` a nie sú registrované v unique tabuľke. Dôvodom je, že terminály sú vždy práve dva a zdieľajú sa v celom grafe.

3 Opis funkcií

3.1 Operácie nad unique tabuľkou

3.1.1 `ut_hash`

Hashovacia funkcia kombinuje trojicu (`variable`, `high`, `low`) do jedného 64-bitového čísla pomocou FNV-inšpirovaného algoritmu. Pointery na uzly sú pretypované na celé číslo (`size_t`), čím sa využíva fakt, že pamäťové adresy sú v danom behu programu unikátne a

rovnomerne rozložené. Výsledok je modulárne zredukovaný na veľkosť tabuľky so zárukou nezápornosti.

```
static long long ut_hash(UniqueTable *ut, char variable,
                        BDDNode *high, BDDNode *low) {
    long long h = (unsigned char)variable;
    h = h * 1000003LL ^ (long long)(size_t)high;
    h = h * 1000003LL ^ (long long)(size_t)low;
    return ((h % ut->size) + ut->size) % ut->size;
}
```

3.1.2 ut_lookup

Prehľadá reťazec v príslušnom buckete a vráti existujúci uzol, alebo `NULL` ak uzol ešte neexistuje. Porovnanie je striktné, musia sa zhodovať všetky tri polia kľúča.

3.1.3 ut_get_or_insert

Ide o hlavnú funkciu unique tabuľky. Najprv zavolá `ut_lookup`; ak nájde existujúci uzol, vráti ho bez alokácie. Ak uzol neexistuje, alokuje nový `BDDNode`, vloží ho do tabuľky a skontroluje load factor. Ak je príliš vysoký, spustí rehashing. Táto funkcia je zodpovedná za realizáciu **pravidla merging**. Zabezpečuje, že v BDD nikdy neexistujú dva štrukturálne identické vnútorné uzly.

3.1.4 ut_rehash

Zdvojnásobí pole bucketov a prerozmiestni všetky existujúce záznamy. Záznamy `UTNode` sa nezahadzujú, len sa zmení ich pozícia v poli. Pôvodné pole sa uvoľní. Rehashing je spúšťaný automaticky z `ut_get_or_insert` vždy, keď load factor presiahne 0,75.

3.1.5 ut_free

Prechádza všetky buckety a reťazce, uvoľňuje každý `UTNode` aj jeho `BDDNode`. Terminálne uzly sú uvoľnené osobitne v `BDD_free`, preto nie sú v tabuľke registrované.

3.2 Parsovanie a zjednodušovanie výrazu

3.2.1 eval_term a eval_expression

Dvojica funkcií `eval_term` / `eval_expression` slúži výlučne na overenie správnosti BDD v testoch. Funkcia `eval_term` vyhodnotí jeden konjunktívny term pre konkrétne hodnoty premenných zadane ako reťazec znakov '0'/'1'. Funkcia `eval_expression` iteruje cez termy oddelené znakom '+' a vráti 1, ak aspoň jeden term je splnený (OR). Tieto funkcie nie sú používané pri samotnom budovaní BDD.

3.2.2 simplify

Toto je algoritmicky najdôležitejšia pomocná funkcia. Dostane reťazec reprezentujúci DNF výraz, premennú ako znak a hodnotu (0 alebo 1), a vráti nový výraz vzniknutý dosadením tejto hodnoty za premennú. Princíp je nasledovný:

Pre každý term v DNF sa prechádzajú literály. Ak literál obsahuje danú premennú, dosadí sa jej hodnota, prípadne opačná hodnota pri operátore '!'. Ak literál s touto hodnotou dáva 0,

celý term sa vylúči (AND s 0 je 0). Ak je hodnota 1, literál sa z termu jednoducho odstráni (je splnený, ďalej neovplyvňuje výsledok). Ostatné literály z iných premenných ostávajú v terme nezmenené.

Špeciálny prípad nastáva, keď po vylúčení literálu je zvyšný term prázdny. To znamená, že všetky jeho literály boli splnené, a teda celý výraz má hodnotu 1. Funkcia vtedy okamžite vracia reťazec "1". Ak po spracovaní všetkých termov nezostal žiadny, výsledok je "0".

```
/* Príklad: simplify("A.!B.C+!A.B", 'A', 1) */
/* term 1 A.!B.C: A=1 -> literál A splnený, ostáva "!B.C" */
/* term 2 !A.B: A=1 -> literál !A = 0 -> term vypadáva */
/* Výsledok: "!B.C" */
```

Funkcia pracuje v jednom prechode cez vstupný reťazec a výstupný buffer je alokovaný s dĺžkou vstupného reťazca, čo je horná hranica. Výsledok môže byť len rovnako dlhý alebo kratší. Tým sa predchádza prepísaniu pamäte.

3.3 Rekurzívne budovanie BDD

3.3.1 build

Funkcia `build` implementuje Shannonovu expanziu kombinovanú s redukčnými pravidlami priebežne pri každom rekurzívnom volaní. Dostáva aktuálny výraz, poradie premenných a hĺbku rekurzie (index aktuálnej premennej).

Najprv sa skontrolujú terminálne podmienky: ak je výraz "0" alebo "1", vráti sa príslušný terminálny uzol. Inak sa vyberie aktuálna premenná podľa hĺbky, výraz sa zjednoduší pre oba prípady dosadenia (`var=1` a `var=0`), a rekurzívne sa vybudujú oba podstromy.

```
static BDDNode *build(BDD *bdd, const char *expr,
                      const char *order, int depth) {
    if (strcmp(expr, "0") == 0) return bdd->terminal0;
    if (strcmp(expr, "1") == 0) return bdd->terminal1;

    char var = order[depth];
    char *expr_high = simplify(expr, var, 1);
    char *expr_low = simplify(expr, var, 0);

    BDDNode *high = build(bdd, expr_high, order, depth + 1);
    BDDNode *low = build(bdd, expr_low, order, depth + 1);
    free(expr_high); free(expr_low);

    if (high == low) return high; /* pravidlo eliminácie */
    return ut_get_or_insert(bdd->ut, var, high, low,
                           &bdd->numNodes); /* pravidlo merging */
}
```

Po návrate z rekurzie sa aplikujú dve redukčné pravidlá:

Pravidlo	Popis
Eliminácia	Ak $high == low$ (oba potomkovia sú rovnaký uzol), aktuálny uzol je zbytočný. Funkcia namiesto neho vráti priamo potomka. Uzol sa teda vôbec nevytvorí.
Merging	Ak uzol s rovnakou trojicou (var, high, low) už existuje v unique tabuľke, nový uzol sa nevytvorí. Vráti sa referencia na existujúci. Tým sa dosahuje maximálne zdieľanie uzlov.

Vďaka tomu, že obe pravidlá sú aplikované *pri každom rekurzívnom návrate* a nie až na záver, výsledný BDD je vždy plne redukovaný. Správnosť vyplýva priamo z vlastností Shannonovej expanzie: výraz $f(x_1, \dots, x_n)$ je ekvivalentný $(x_i \wedge f|_{x_i=1}) \vee (\neg x_i \wedge f|_{x_i=0})$, pričom dosadenie vykoná funkcia `simplify` presne podľa tohto vzťahu.

3.4 Rozhranie implementácie

3.4.1 BDD_create

Alokuje štruktúru `BDD`, inicializuje unique tabuľku, vytvorí oba terminálne uzly a zavolá `build` na koreni rekurzie. Terminálne uzly sú nastavené priamo bez vloženia do unique tabuľky, pretože sa na ne odkazuje priamo z koreňa `BDD` a nesmú byť uvoľnené skôr ako celý graf.

3.4.2 BDD_create_with_best_order

Funkcia hľadá kompaktné poradie premenných pomocou dvojfázovej heuristiky: frequency sort a sifting.

Frequency sort: Funkcia najprv spočíta, koľkokrát sa každá premenná vyskytuje vo výraze (počet výskytov znakov). Premenné sa zoradia zostupne podľa tejto frekvencie. Myšlienkou je, že premenné rozhodujúce o väčšom počte termov by mali byť bližšie ku koreňu BDD, kde ich rozvetvenie eliminuje čo najviac podstromov.

Sifting: Zo zoradeného poradia sa skonštruuje počiatočný BDD. Následne sa pre každú premennú (iterácia cez index i) skúsi posunúť ju na všetky pozície doľava susedným swapom, potom sa order resetuje na doposiaľ najlepšie poradie a premenná sa skúsi posunúť na všetky pozície doprava. Po každom swape sa skonštruuje nový BDD a porovná sa počet uzlov s doterajším minimom. Ak swap zlepšil výsledok, nové poradie sa uloží ako aktuálne najlepšie. Po prehľadaní oboch smerov sa order znova resetuje na najlepšie nájdené poradie a pokračuje sa ďalšou premennou. Celý prechod cez všetky premenné sa opakuje, kým dochádza k zlepšeniu; počet volaní `BDD_create` je navyše obmedzený hodnotou $2 \cdot n$, kde n je počet premenných, čím sa zaručuje rozumný čas aj pre väčšie n .

3.4.3 BDD_use

Vyhodnotí BDD pre konkrétny vstupný vektor. Parameter `inputs` musí obsahovať hodnoty premenných v poradí zhodnom s `bdd->varOrder`, nie nevyhnutne v abecednom poradí. Napríklad ak `varOrder = "CAB"`, potom `inputs[0]` je hodnota C, `inputs[1]` je hodnota A a `inputs[2]` je hodnota B. Funkcia postupuje od koreňa smerom k listu: v každom uzle našla index aktuálnej premennej v `varOrder` a podľa zodpovedajúceho bitu vstupu sa pohne na

vetvu high alebo low. Keď dosiahne terminálny uzol (rozpoznaný podľa `variable == '\0'`), vráti jeho hodnotu ako znak '0' alebo '1'. Funkcia má lineárnu zložitosť vzhľadom na počet premenných, resp. hĺbku BDD.

3.4.4 BDD_free

Uvoľní celý BDD: zavolá `ut_free` (ktorá uvoľní všetky `BDDNode` a `UTNode`), potom osobitne uvoľní oba terminálne uzly, reťazec `varOrder` a nakoniec samotnú štruktúru `BDD`. Terminálne uzly musia byť uvoľnené až po `ut_free`, pretože na ne môžu odkazovať aj záznamy v tabuľke. Hoci v tejto implementácii tam registrované nie sú, poradie uvoľnenia je tak správne aj pre prípadné rozšírenia.

4 Spôsob testovania

4.1 Filozofia a ciele testovania

Testovanie implementácie BDD bolo navrhnuté tak, aby pokrylo dve základné vlastnosti riešenia: **korektnosť** a **efektívnosť**. Samotná správnosť výsledku je primárna. Efektívnosť hovorí o tom, nakoľko je diagram zredukovaný oproti plnému rozhodovaciemu stromu, čo priamo ovplyvňuje pamäťovú a časovú náročnosť pri praktickom použití.

Testovanie je plne automatizované a deterministické. Seed generátora náhodných čísel je fixne nastavený na hodnotu `42`, čo zaručuje reprodukovateľnosť každého testu. Pre každú skupinu premenných je vygenerovaných **100 náhodných booleovských funkcií** v DNF tvare, pričom každá funkcia je overovaná dvakrát. Raz pre funkciu `BDD_create` (pevné poradie premenných) a raz pre `BDD_create_with_best_order` (hľadanie optimálneho poradia). Celkovo tester vykoná **1 400 nezávislých overení** naprieč siedmimi skupinami veľkostí vstupu.

4.2 Štruktúra testovacieho programu

Testovací program `test_bdd.c` je napísaný čisto v jazyku C a nevyžaduje žiadne externé knižnice. Stačí ho skompilovať spolu s implementáciou:

```
gcc -O2 -o test_bdd test_bdd.c bdd.c
./test_bdd
```

Program je logicky rozdelený do troch fáz, ktoré na seba nadväzujú:

Fáza 1: Generovanie vstupu

Funkcia `random_dnf()` pre zadaný počet premenných n vygeneruje náhodný DNF výraz nad abecedou $\{A, B, \dots, A+n-1\}$. Počet termov je $n/2$ až $2n$, čo pri väčších n zaručuje dostatočné pokrytie priestoru premenných. Každá premenná sa do každého termu zaradí s pravdepodobnosťou 50 % (s náhodnou negáciou). Ak by term zostal prázdny, doplní sa aspoň jeden náhodný literál. Záchranný mechanizmus na konci garantuje, že každá z n premenných sa vyskytne aspoň raz v celom výraze.

Fáza 2 – Overenie korektnosti

Funkcia `verify_bdd()` prechádza všetkými 2^n vstupnými kombináciami. Pre každú kombináciu vypočíta referenčnú hodnotu priamym vyhodnotením výrazu cez `eval_expression()` a porovná ju s výstupom `BDD_use()`. Obe funkcie musia vrátiť identický výsledok.

Fáza 3 – Meranie a štatistiky

Pre každý test sa meria čas konštrukcie BDD (v milisekundách), počet uzlov výsledného diagramu, miera redukcie voči plnému rozhodovaciemu stromu a percentuálna extra redukcia dosiahnutá optimálnym poradím oproti lexikografickému poriadku.

4.3 Generovanie náhodných DNF funkcií

Testovanie využíva náhodné DNF (disjunktívna normálna forma) funkcie namiesto ručne písaných prípadov z niekoľkých dôvodov. Ručne písané testy pokrývajú iba scenáre, na ktoré autor myslel; náhodné testovanie systematicky odhaľuje okrajové prípady, na ktoré sa pri návrhu nemyslelo. Pri 100 funkciách na každú skupinu a plnom prehľade 2^n kombinácií je šanca na prehladnutie chyby extrémne nízka.

Kľúčovou požiadavkou na generátor je, že vygenerovaný výraz musí skutočne závisieť od **všetkých n premenných**. Bez tohto by pri $n=15$ mohol generátor vytvoriť výraz závislý len od 3 - 5 premenných, čo by testy pre väčšie n degradovalo na testy malých funkcií. Generátor to rieši dvojstupňovo: veľký počet termov ($n/2$ až $2n$) štatisticky zaručuje, že každá premenná sa prirodzene objaví a záchranný mechanizmus následne explicitne doplní ľubovoľnú premennú, ktorá by náhodou chýbala.

Algoritmus generovania jedného DNF výrazu funguje nasledovne:

```
/* Príklad vygenerovanej funkcie pre n=5: */
/* "A.!C + !B.D + !B.C.!E + B + !E"      */

int min_terms = n / 2;
int num_terms = min_terms + rand() % (2*n - min_terms + 1); /* n/2 az
2n */
```



```

int used[MAX_VARS] = {0};

for (int t = 0; t < num_terms; t++) {
    for (int i = 0; i < n; i++) {
        if (rand() % 2 == 0) continue;    /* 50% sanca zahrnutia */
        if (rand() % 2 == 0) buf[pos++] = '!';
        buf[pos++] = 'A' + i;
        used[i] = 1;
    }
}
/* Zaruci ze kazda premenna sa vyskytne aspon raz */
for (int i = 0; i < n; i++) {
    if (!used[i]) { buf[pos++] = '+'; buf[pos++] = 'A' + i; }
}

```

Pravdepodobnosť, že záchranný mechanizmus musí zasiahnuť pre konkrétnu premennú, je $(0.5)^{(n/2)}$. Pre $n=15$ je to $(0.5)^7 \approx 0.8\%$. Záchranný mechanizmus teda zasahuje len výnimočne a výrazy sú prevažne prirodzene vygenerované. Výrazy tak pokrývajú celé spektrum zložitosti, od jednoduchých termov s malým počtom literálov, až po husté termy zahŕňajúce väčšinu premenných naraz.

4.4 Verifikačná metóda – úplný prehľad vstupov

Kľúčovým rozhodnutím pri návrhu testovania je použitie **úplného prehľadu** (angl. exhaustive testing) namiesto vzorkovania. Pre každý vygenerovaný výraz sa otestujú všetky 2^n vstupných kombinácií, nie len niekoľko náhodne vybraných. Pre $n = 15$ to znamená overenie 32 768 kombinácií na jeden výraz, teda 3 276 800 individuálnych overení pre celú skupinu.

Verifikácia stojí na porovnaní dvoch nezávislých ciest výpočtu:

```

/* Referenčná hodnota - priame vyhodnotenie výrazu */
char expected = eval_expression(expr, var_order, canonical) ? '1' :
'0';

/* Testovaná hodnota - prechod BDD grafu */
char got = BDD_use(bdd, bdd_inputs);

/* Obe musia byť identické pre každú vstupnú kombináciu */
if (got != expected) errors++;

```

Funkcia `eval_expression()` slúži ako **referenčná implementácia**. Vyhodnocuje DNF výraz priamym prechodom bez akejkoľvek optimalizácie. Je zámerné, že ide o čo najjednoduchší kód bez zdieľania uzlov, memoizácie ani iných optimalizácií prítomných v BDD. Tým pádom

sú obe implementácie navzájom nezávislé, a ak obe vrátia rovnaký výsledok, dáva nám to vysokú istotu o správnosti BDD.

Funkcia `BDD_create_with_best_order` sa overuje rovnakým spôsobom ako `BDD_create`. Funkcia `verify_bdd()` vždy premapuje vstupný vektor z abecedného poradia do poradia `bdd->varOrder`, pretože `BDD_use()` očakáva vstupy práve v tomto poradí. Premapovanie funguje tak, že pre každú pozíciu v `varOrder` sa nájde pozícia tej istej premennej v abecednom poradí a skopíruje sa zodpovedajúca hodnota. Premapovanie je súčasťou `verify_bdd()` vždy, bez ohľadu na to, či bol BDD vytvorený cez `BDD_create` alebo `BDD_create_with_best_order`. Pri lexikografickom poradí je výsledok premapovania totožný so vstupom, no kód sa nijak nerozlišuje.

4.5 Skupiny premenných a rozsah testovania

Testy sú organizované do siedmich skupín podľa počtu premenných booleovskej funkcie. Skupiny pokrývajú rozsah od triviálnych prípadov ($n = 3$) až po reálne zaťažujúce prípady ($n = 15$), kde plný rozhodovací strom má 32 767 vnútorných uzlov. Stĺpec „Počet kombinácií (2^n)“ udáva, koľko rôznych vstupných vektorov existuje pre daný počet premenných, teda koľko prípadov musí funkcia `verify_bdd()` pre každý vygenerovaný výraz otestovať. Keďže testovanie používa úplný prehľad (nie vzorkovanie), každý z týchto vstupov je skutočne overený:

Počet premenných (n)	Počet kombinácií (2^n)	Počet funkcií na skupinu
3	8	100
5	32	100
7	128	100
9	512	100
11	2 048	100
13	8 192	100
15	32 768	100

Voľba tohto rozsahu nie je náhodná. Pre malé n (3 - 7) je možné výsledky jednoducho overovať aj manuálne a slúžia na overenie základnej správnosti. Pre stredné n (9 - 11) sa naplno prejavuje efekt redukcie. Diagram je výrazne menší ako plný strom. Pre veľké n (13 - 15) sa testuje škálovateľnosť implementácie a meria sa čas vykonania, ktorý je pri naivných implementáciách exponenciálny.

4.6 Merané metriky

Pre každú skupinu sú zaznamenávané nasledujúce metriky:

Metrika	Popis a spôsob výpočtu
Uzly avg (BDD_create)	Priemerný počet vnútorných (ne-terminálnych) uzlov výsledného BDD pri lexikografickom poradí premenných. Čím menej uzlov, tým lepšia redukcia.
Uzly avg (BDD_best_order)	Rovnaká metrika pre BDD s poradím premenných hľadaným cez frequency sort a sifting.
Redukcia %	Percentuálna miera redukcie oproti plnému rozhodovaciemu stromu: $(\text{plný} - \text{BDD}) / \text{plný} \times 100$. Plný strom pre n premenných má $2^n - 1$ uzlov.
Extra redukcia %	Percentuálna extra redukcia dosiahnutá optimálnym poradím oproti lexikografickému: $(\text{uzly_create} - \text{uzly_best}) / \text{uzly_create} \times 100$. Kladná hodnota znamená, že best_order vyhral.
Best < Create	Počet prípadov, kedy BDD_best_order dosiahol menší počet uzlov ako BDD_create. Maximálne môže byť 100.
Čas BDD_create (ms)	Celkový čas vykonania BDD_create pre všetkých 100 funkcií v danej skupine. Meria sa pomocou CLOCK_MONOTONIC na Linuxe a QueryPerformanceCounter na Windows.
Čas BDD_best_order (ms)	Celkový čas vykonania BDD_create_with_best_order vrátane frequency sortu a siftingu.

4.7 Meranie času a platformová nezávislosť

Čas vykonania je meraný na úrovni celej skupiny (100 funkcií), nie pre každý test zvlášť. Dôvodom je presnosť. Pre malé n (3 - 5) je konštrukcia jedného BDD rýchlejšia ako rozlišovacia schopnosť systémových hodín, a individuálne meranie by prinieslo len šum. Súčet cez celú skupinu dáva stabilné a porovnateľné čísla.

Meranie je implementované platformovo nezávisle:

```
#ifdef _WIN32
/* Windows: QueryPerformanceCounter (vysoká presnosť) */
static double get_time_ms(void) {
    LARGE_INTEGER freq, cnt;
    QueryPerformanceFrequency(&freq);
    QueryPerformanceCounter(&cnt);
```

```

        return (double)cnt.QuadPart / freq.QuadPart * 1000.0;
    }
    #else
    /* Linux/macOS: clock_gettime(CLOCK_MONOTONIC) */
    static double get_time_ms(void) {
        struct timespec ts;
        clock_gettime(CLOCK_MONOTONIC, &ts);
        return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
    }
    #endif

```

Použitie `CLOCK_MONOTONIC` namiesto `clock()` je dôležité; `clock()` meria procesorový čas a nie je vhodný pre benchmarky, pretože môže byť ovplyvnený plánovaním procesov. Monotónne hodiny merajú skutočne uplynulý čas od štartu systému a nie sú náchylné na záporné skoky ani ovplyvnenie systémovým časom.

4.8 Záver - prečo je takéto testovanie dostatočné

Kombinácia úplného prehľadu vstupov s nezávislou overovacou referenčnou implementáciou a deterministickým generátorom poskytuje dostatočné pokrytie na odhalenie väčšiny implementačných chýb. Pre každú z 700 testovaných funkcií je garantované, že BDD vyhodnocuje správne *každú jednu* vstupnú kombináciu. Ak testy prejdú s 0 chybami, môžeme s vysokou mierou dôvery tvrdiť, že implementácia Shannon expansion, redukčné pravidlá (elimination a merging cez unique table) aj premapovanie vstupov v `BDD_use()` fungujú správne.

Testovanie **nie je** len syntaktickým overením. Funkcia `eval_expression()` je napísaná úplne iným spôsobom ako BDD (lineárny prechod výrazom bez rekurzie, bez grafu), takže ak obe vrátia rovnaký výsledok, ide o silný dôkaz korektnosti, nie len o konzistenciu dvoch implementácií toho istého algoritmu.

Súhrn parametrov testovania

Skupiny n: 3, 5, 7, 9, 11, 13, 15

Funkcií/skupina: 100

Celkovo testov: $700 \times 2 = 1\,400$

Seed: 42 (fixný, reprodukovateľný)

Max. overení (n=15): $100 \times 32\,768 = 3\,276\,800$ kombinácií

5 Výsledky testovania

5.1 Tabuľka výsledkov

n	nTest	Správne (create)	Správne (best)	Uzly avg (create)	Uzly avg (best)	Redukcia %	Extra redukcia %	Best < Create	t_create ms / t_best ms
3	100	100	100	2,31	2,18	67,00 %	3,05 %	11/100	0,12 / 0,90
5	100	100	100	6,17	4,99	80,10 %	14,52 %	58/100	0,41 / 9,70
7	100	100	100	16,81	11,46	86,76 %	29,98 %	94/100	1,31 / 80,39
9	100	100	100	42,60	27,94	91,66 %	34,77 %	99/100	4,05 / 533,09
11	100	100	100	96,01	63,10	95,31 %	35,67 %	100/100	12,87 / 2 984,15
13	100	100	100	196,02	120,69	97,61 %	39,30 %	100/100	42,29 / 13 688,82
15	100	100	100	351,86	210,03	98,93 %	41,17 %	100/100	110,53 / 54 896,16

Stĺpce: Redukcia % – percento uzlov ušetrených oproti plnému binárneho stromu ($2^n - 1$ uzlov). Extra redukcia % – percento uzlov ušetrených použitím BDD_create_with_best_order oproti BDD_create. Best < Create – počet prípadov, kde best_order skutočne dosiahol menší počet uzlov.

5.2 Detailná štatistika extra redukcie (best vs. create)

n	Min	Priemer	Max
3	0,00 %	3,05 %	40,00 %
5	0,00 %	14,52 %	45,45 %
7	0,00 %	29,98 %	58,82 %
9	0,00 %	34,77 %	61,54 %
11	2,44 %	35,67 %	69,31 %
13	9,33 %	39,30 %	65,79 %
15	18,92 %	41,17 %	62,57 %

5.3 Zhrnutie

Všetkých 700 testov prešlo bez chyby pre obe funkcie (100,0 %). S rastúcim n sa efekt optimálneho usporiadania premenných výrazne zvyšuje – pri n = 15 dosiahla priemerná extra redukcia 41,17 % a vo všetkých 100 prípadoch bol BDD s najlepším poradím menší ako ten s lexikografickým. Cena za toto zlepšenie je výrazne vyšší čas výpočtu: zatiaľ čo BDD_create spracuje skupinu n = 15 za ~110 ms, BDD_create_with_best_order potrebuje ~55 sekúnd.

6 Odhad výpočtovej a pamäťovej zložitosti

6.1 BDD_create

Časová zložitosť

Bez hašovacej tabuľky by redukcia typu I (merging) vyžadovala porovnanie všetkých dvoíc uzlov na každej úrovni. Pre M uzlov na jednej úrovni je to $O(M^2)$, pričom $M \leq 2^N$, čo dáva celkovú zložitosť $O((2^N)^2)$.

Implementácia však používa unique table (hašovaciu tabuľku s reťazením), kde kľúčom je trojica (premenná, high*, low*). Vďaka tomu sa porovnávajú len uzly v rámci toho istého bucketu. Za predpokladu vhodnej hašovacej funkcie a veľkosti tabuľky je zložitosť redukcie na jednej úrovni $O(M)$ namiesto $O(M^2)$. Celkový počet uzlov cez všetky úrovne je ohraničený veľkosťou výsledného BDD, teda $|BDD| \leq 2^N$.

Celková časová zložitosť: $O(2^N)$ v najhoršom prípade, v praxi výrazne menej vďaka redukcii.

Priestorová zložitosť

Unique table uchováva každý uzol práve raz. Počet uzlov je $|BDD| \leq 2^N$.

Celková priestorová zložitosť: $O(2^N)$

6.2 BDD_create_with_best_order

Časová zložitosť

Funkcia opakuje volania BDD_create pre rôzne poradie premenných. Sifting pre jednu premennú na pozícii i vyskúša i pozícií doľava a $(n-1-i)$ doprava, teda $O(n)$ volaní BDD_create na premennú. Pre všetkých n premenných v jednej iterácii vonkajšieho cyklu je to $O(n^2)$ volaní, pričom vonkajší cyklus beží najviac n^2 iterácií.

Každé volanie BDD_create stojí $O(2^N)$, takže:

Celková časová zložitosť: $O(n^2 \cdot 2^N)$ pre jednu iteráciu siftingu, resp. $O(n^4 \cdot 2^N)$ v absolútnom najhoršom prípade (všetky iterácie vonkajšieho cyklu).

Priestorová zložitosť

V každom okamihu existujú v pamäti súčasne najviac dve BDD – aktuálny kandidát a doteraz najlepší. Ostatné sa ihneď uvoľnia.

Celková priestorová zložitosť: $O(2^N)$